

[← Go Back](#)

Hardware

Nicla Sense Env



Tutorials

[Nicla Sense Env User Manual](#)[Monitoring Outdoor Air Quality with the Nicla Sense Env](#)[Smart Elevator Monitoring System with the Portenta Proto Kit](#)

Home / Hardware / Nicla Sense Env / [Monitoring Outdoor Air Quality with the Nicla Sense Env](#)

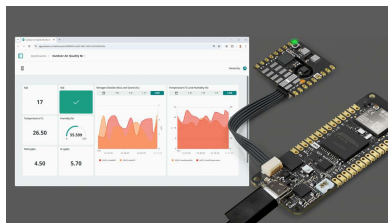
Monitoring Outdoor Air Quality with the Nicla Sense Env

This application note describes how to implement a simple outdoor air quality monitor with the Arduino® Nicla Sense Env.

Author · José Bagur · Last revision · 10/10/2024

Introduction

Air pollution is a growing concern in urban and industrial areas due to the harmful effects of airborne pollutants such as nitrogen dioxide (NO₂) and ozone (O₃) on human health and the environment. This application note describes building a basic outdoor air quality monitor using the Arduino® Portenta C33 and the Nicla Sense Env board.



The outdoor air quality monitor provides real-time data on temperature (°C), humidity (%),

ON THIS PAGE

Introduction

[Goals](#)[Hardware and Software Requirements](#) +[Hardware Setup Overview](#)[Understanding Outdoor Air Quality](#) +[Simple Environmental Monitor Example Sketch](#) +[Improving the Simple Environmental Monitor Example Sketch](#) +[Connecting the Environmental Monitor to Arduino Cloud](#) +[Conclusions](#)[Next Steps](#)[Help](#)

Air Quality Index (AQI). It is suitable for deployment in urban areas, industrial zones or environmental research.

Goals

The main goals of this application note are the following:

- Develop and implement a simple outdoor air quality monitor that provides real-time data on temperature, humidity, and pollutants such as nitrogen dioxide (NO₂) and ozone (O₃).

- Collect real-time data on NO₂, O₃ and the outdoor Air Quality Index (AQI) to evaluate outdoor air conditions.

- Display live readings from the Nicla Sense Env board using the Arduino IDE Serial Monitor for immediate analysis.

- Use the RGB LED on the Nicla Sense Env board to provide visual feedback based on the AQI, indicating air quality levels with different colors.

- Enable remote monitoring and analysis by connecting the outdoor air quality monitor to the Arduino Cloud.

Hardware and Software Requirements

Hardware Requirements

[Nicla Sense Env](#) (x1)

[Portenta C33](#) (x1)

[USB-C® cable](#) (x1)

Wi-Fi® W.FL antenna (x1)

Software Requirements

[Arduino IDE 2.0+](#) or

[Arduino Web Editor](#)

[Arduino_NiclaSenseEnv library](#)

[Arduino Renesas](#)

[Portenta Boards core](#)

(required to work with the Portenta C33 board)

For the Wi-Fi® connectivity feature of Portenta C33, we will use the [Arduino Cloud](#). If you do not have an account, create one for free [here](#).

The Nicla Sense Env board is not intended as a standalone device but as a shield of another Portenta, MKR, or Nano

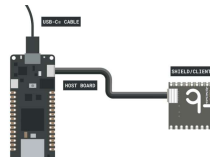


family board. In this application note, we will use the Portenta C33 as the main board and the Nicla Sense Env board as a shield.

Hardware Setup Overview

The electrical connections for

are outlined in the diagram below:



The outdoor air quality monitor hardware setup

This diagram shows how the components are connected. The **Portenta C33** is the **host board** or primary controller, while the **Nicla Sense Env** board (the **client board** or shield) collects temperature, humidity, and outdoor air quality data.

The Nicla Sense Env board connects to the Portenta C33 board using the **ESLOV interface**. The outdoor air quality monitor operates with a single power supply bus (+5 VDC from the Portenta C33), which powers the Nicla Sense Env. The Portenta C33 is powered via its onboard USB-C port for testing purposes.

Understanding Outdoor Air Quality

Air pollution is not just a problem in urban areas; rural environments also face significant risks from pollutants like nitrogen dioxide (NO₂) and

for vulnerable populations. NO₂, produced by combustion engines and industrial processes, can travel far beyond urban centers. O₃, formed by atmospheric chemical reactions, affects urban and rural regions, especially during warmer seasons of the year.

Real-time monitoring is critical in urban and rural settings to properly manage exposure to these pollutants. Air quality tracking and monitoring help individuals and communities proactively reduce health risks.

The Role of Sensors in Air Quality Monitoring

This application note uses advanced sensors to provide accurate, real-time measurements of various pollutants. The Nicla Sense Env board offers a versatile outdoor air quality monitoring solution.

NO₂ Sensor: Detects nitrogen dioxide, a harmful gas released by vehicles and industrial activities.

O₃ Sensor: Measures ozone, a gas that forms in the atmosphere through reactions between pollutants and sunlight and contributes to respiratory problems.

Outdoor Air Quality Index (AQI): The outdoor AQI simplifies the interpretation of air quality by combining data

The AQI scale ranges from 0 to 500, with the following classifications:

AQI Level		Health Recommendation
Good	0-50	Air quality is considered satisfactory, and air pollution poses little or no risk.
Moderate	51-100	Acceptable, but there may be some concern for unusually sensitive people.
Unhealthy for Sensitive Groups	101-150	Members of sensitive groups may experience health effects. The general public is not likely to be affected.
Unhealthy	151-200	Some members of the general public may experience health effects; the general public is more likely to be affected.
Very Unhealthy	201-300	Air quality is hazardous. Everyone may experience more serious health effects.
Hazardous	301-500	Health warnings of emergency conditions. The entire population is more likely to be affected.

AQI scale ranges and health advice

The AQI makes it easy to understand how outdoor air quality affects public health, particularly when pollutant levels reach unhealthy or hazardous thresholds.

Monitoring NO₂, O₃, and AQI in real-time allows for immediate responses when air quality worsens. This is especially important in rural areas with less visible air quality issues. The data collected can be sent to platforms like Arduino Cloud, enabling remote monitoring and long-term trend analysis, improving public awareness and policy decisions.

Simple Environmental Monitor Example Sketch

Now that we have covered the hardware components of our environmental monitor and how they are interconnected, let's

this monitor to life. The example sketch in the following section manages data collection from the Nicla Sense Env board connected to the Portenta C33 board, allowing us to monitor real-time environmental data such as temperature, humidity and outdoor air quality.

This section breaks down the provided example sketch, guiding you through its functionality. We will explore how the sensors are initialized, how the data is collected every 10 seconds and how the results are displayed in the Arduino IDE's Serial Monitor.

The complete example sketch is shown below.



```
1  /**
2   Outdoor Air Quality
3   Name: outdoor_air
4   Purpose: This sketch
5   from a Nicla Sense
6   reported to the A
7
8   @version 1.0 06/10
9   @author Arduino P
10 */
11
12 // Include the neces
13 #include "Arduino_N
14
15 // Time interval (in
16 static const uint32
17 uint32_t lastReadTim
18
19 // Sensor object for
20 NiclaSenseEnv device
21
22 /**
23   Initializes the se
24   Configures the Nic
25   temperature/humid
26   quality sensor.
27 */
28 void setup() {
```

The following sections will help you to understand the main parts of the example sketch shown before, which can be divided into the following:

- Library imports
- Sensors initialization
- Data collection
- Data display

Library Imports

The first step is to ensure that all necessary libraries are included to control the Nicla Sense Env board. These libraries provide all the functionality to

communicate with and extract sensor data.

```
1 // Include the necessary libraries
2 #include "Arduino_NiclaSenseEnv.h"
3
4 // Time interval (in milliseconds)
5 static const uint32_t READ_INTERVAL = 10000;
6 uint32_t lastReadTime = 0;
7
8 // Sensor object for NiclaSenseEnv
9 NiclaSenseEnv device;
```

In the code snippet shown above:

The

`Arduino_NiclaSenseEnv`

library is included to provide access to the onboard sensors.

The constant

`READ_INTERVAL` defines

the time between each sensor reading (10 seconds in this example).

An instance of the

`NiclaSenseEnv` class is

created to manage the onboard sensors.

Sensors Initialization

The `setup()` function

initializes the monitor sensors and prepares them to start collecting data. This is where the sensors "wake up" and prepare to make measurements.

```
1 void setup() {  
2     // Initialize ser  
3     Serial.begin(1152  
4  
5     // Wait for Serial  
6     for(uint32_t star  
7  
8     // Initialize Nic  
9     if (device.begin(  
10        Serial.print  
11  
12        // Enable the  
13        device.outdoc  
14    } else {  
15        // Error mess  
16        Serial.print  
17    }  
18 }
```

In the code snippet shown above:

Serial communication is initialized at a `115200` baud rate, allowing data to be sent to the IDE's Serial Monitor.

The Nicla Sense Env board is initialized to read temperature, humidity and the outdoor Air Quality Index (AQI).

Data Collection

Once the sensors are ready, the example sketch must continuously collect data. The `loop()` function ensures that data is captured every 10 seconds and sent to the IDE's Serial Monitor.

```
1 void loop() {  
2     // Get the current  
3     uint32_t currentTime  
4  
5     // Read sensors e  
6     if (currentTime -  
7         lastReadTime  
8  
9         // Read and c  
10    displayAllData  
11    }  
12 }
```

In the code snippet shown above:

The `loop()` function continuously checks if the interval (10 seconds) has passed since the last reading.

Once the interval is reached, the `displayAllData()` function is called to read sensor data.

Data Display

The `displayAllData()` function pulls together the data from each sensor and displays it in a single line on the IDE's Serial Monitor. This is the most dynamic part of the example sketch, where the sensors provide real-time readings.

```
1 void displayAllData
2     // Get the temp
3     TemperatureHumid
4     OutdoorAirQuali
5
6     // Check if botl
7     if (tempHumSens
8         // Read data
9         float tempe
10        float humid:
11        float NO2 =
12        float O3 =
13        int airQual:
14
15        // Print all
16        Serial.prin
17        Serial.prin
18        Serial.prin
19        Serial.prin
20        Serial.prin
21        Serial.prin
22        Serial.prin
23        Serial.prin
24        Serial.prin
25        Serial.prin
26    } else {
27        // Error me
28        Serial.prin
```

In the code snippet shown
before:

The function first checks
that the
temperature/humidity
sensor and the outdoor
air quality sensor are
enabled.

The function retrieves
temperature, humidity,
gas concentrations (NO₂
and O₃), and the outdoor
Air Quality Index (AQI).

Finally, the data is printed
in a single line on the IDE's
Serial Monitor, with the
AQI displayed at the end.

After uploading the example
sketch to the host board, you

in the Arduino IDE's Serial Monitor:



Example sketch
output in the Arduino
IDE's Serial Monitor

Complete Example Sketch

The complete improved example sketch can be downloaded [here](#).

Improving the Simple Environmental Monitor Example Sketch

The previous section explored setting up a simple environmental monitor using the Nicla Sense Env and the Portenta C33 board. While the example sketch provided detailed readings on temperature, humidity and air quality, we can improve the example sketch by incorporating a visual indicator for the outdoor Air Quality Index (AQI).

In this section, we will improve the example sketch by adding

on the Nicla Sense Env board.
The RGB LED will visually represent the outdoor AQI, changing colors based on its value.



The complete example sketch is shown below. The RGB LED will change color based on the following:

Green ● for **good air quality**.

Yellow ● for **moderate air quality**.

Red ● for **bad air quality**.

```

1  /**
2   Outdoor Air Qual:
3   Name: outdoor_air
4   Purpose: This sketch
5   from a Nicla Sense
6   onboard RGB LED c
7
8   @version 1.0 06/11/2019
9   @modified by: Arduino
10 */
11
12 // Include the necessary
13 #include "Arduino.h"
14
15 // Time interval (in seconds)
16 static const uint32_t interval = 1000;
17 uint32_t lastReadTime = 0;
18
19 // Sensor object for NiclaSenseEnv device
20 NiclaSenseEnv device;
21
22 /**
23  Initializes the NiclaSenseEnv device.
24  Configures the NiclaSenseEnv device to read
25  temperature/humidity and the RGB LED.
26 */
27 */
28 void setup() {

```

The following sections will help you to understand the main parts of the example sketch shown before, which can be divided into the following:

- Library imports
- Sensors initialization
- Data collection
- Data display

Library Imports

As in the original example sketch, we include the necessary libraries for interacting with the Nicla Sense Env onboard sensors. The RGB

managed through the

`Arduino_NiclaSenseEnv`

library.

```
1 // Include the necessary libraries
2 #include "Arduino_NiclaSenseEnv.h"
3
4 // Time interval (in milliseconds)
5 static const uint32_t READ_INTERVAL = 10000;
6 uint32_t lastReadTime = 0;
7
8 // Sensor object for NiclaSenseEnv
9 NiclaSenseEnv device;
```

In the code snippet shown before:

The

`Arduino_NiclaSenseEnv`

library is included to provide access to the onboard sensors and the RGB LED.

The constant

`READ_INTERVAL` defines

the time between each sensor reading (10 seconds in this example).

An instance of the

`NiclaSenseEnv` class is

created to manage the onboard sensors and the RGB LED.

Sensors Initialization

In the `setup()` function, we initialize the Nicla Sense Env's onboard sensors and RGB LED. The outdoor air quality sensor is activated, and the onboard RGB LED is prepared to display the outdoor AQI visually based on the sensor readings.


```
1 void setup() {  
2     // Initialize serial communication  
3     Serial.begin(115200);  
4  
5     // Wait for Serial communication to start  
6     for(uint32_t start = 0; start < 1000000; start++)  
7     {  
8         // Initialize Nicla Sense Env board  
9         if (device.begin() != 0)  
10            Serial.println("Nicla Sense Env board not found");  
11  
12            // Enable the outdoor Air Quality Index (AQI) sensor  
13            device.outdoor_aqi.begin();  
14  
15            // Enable the RGB LED  
16            Serial.println("RGB LED initialized");  
17        } else {  
18            Serial.println("Nicla Sense Env board not found");  
19        }  
20    }
```

In the code snippet shown before:

Serial communication is initialized at a `115200` baud rate, allowing data to be sent to the IDE's Serial Monitor.

The Nicla Sense Env board is initialized to read temperature, humidity and the outdoor Air Quality Index (AQI).

The onboard RGB LED is also initialized and prepared to be controlled based on the outdoor AQI.

Data Collection

Once the sensors are ready, the example sketch must continuously collect data. The `loop()` function ensures that data is captured every 10 seconds and sent to the IDE's

```
1 void loop() {  
2     uint32_t currentTime  
3  
4     if (currentTime -  
5         lastReadTime ==  
6         displayAllData  
7     }  
8 }
```

In the code snippet shown before:

The `loop()` function continuously checks if the interval (10 seconds) has passed since the last reading.

Once the interval is reached, the `displayAllData()` function is called to read sensor data and update the RGB LED accordingly.

Data Display

The `displayAllData()` function collects temperature, humidity, NO₂, O₃, and outdoor AQI values. Based on the outdoor AQI, it updates the RGB LED color to reflect air quality conditions: green ● for good, yellow ● for moderate, and red ● for unhealthy.

```
1 void displayAllData
2     // Get the temp
3     TemperatureHumid
4     OutdoorAirQuali
5     auto rgbLED = d
6
7     // Check if botl
8     if (tempHumSens
9         // Read data
10        float tempe
11        float humid:
12        float NO2 =
13        float O3 =
14        int airQual:
15
16        // Print all
17        Serial.print
18        Serial.print
19        Serial.print
20        Serial.print
21        Serial.print
22        Serial.print
23        Serial.print
24        Serial.print
25        Serial.print
26        Serial.print
27
28        // Update tl
```

In the code snippet shown
before:

The function first checks
that the
temperature/humidity
sensor and the outdoor
air quality sensor are
enabled.

The function retrieves
temperature, humidity,
gas concentrations (NO₂
and O₃), and the outdoor
Air Quality Index (AQI).

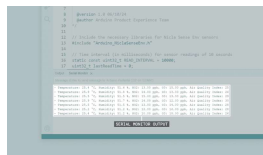
Finally, the data is printed
in a single line on the IDE's
Serial Monitor, with the
outdoor AQI displayed at
the end.

Based on the outdoor AQI,
the RGB LED color is set to

indicate good, moderate, or unhealthy air quality.

The function ensures that the RGB LED brightness is 100% for visibility.

After uploading the example sketch to the host board, you should see the following output in the Arduino IDE's Serial Monitor:



Example sketch output in the Arduino IDE's Serial Monitor

Complete Example Sketch

The complete improved example sketch can be downloaded [here](#).

Connecting the Environmental Monitor to Arduino Cloud

As environmental monitoring continues to grow in importance, connecting devices to the Internet enables real-time, remote data monitoring, allowing for easier tracking and response to changing conditions. Connecting our

the [Arduino Cloud](#) allows us access to live sensor data from anywhere. This connection turns the monitor into an Internet of Things (IoT) device, offering timely insights into air quality, temperature, and humidity through a centralized dashboard.

In this section, we will connect the environmental monitor to Arduino Cloud. The data collected from the monitor's sensors (temperature, humidity, NO₂, O₃, and AQI) will be transmitted to the Arduino Cloud, which can be monitored via a web-based dashboard or a mobile app. The onboard RGB LED of the Portenta C33 will continue to provide visual feedback based on the current AQI level.

 If you are new to Arduino Cloud, please check out [this tutorial](#).

Please begin by creating a new **Thing** in your Arduino Cloud with the following variables:

cloudTemperature
cloudHumidity
cloudNO2
cloudO3
cloudAQI
cloudAQIAlert

All variables must be `float` type except for the `cloudAQIAlert` (`boolean`), have `Read Only` permission

`thingProperties.h` header file, automatically generated by the Arduino Cloud, remember to update your Wi-Fi® credentials (SSID and password).

The complete example sketch is shown below and can be downloaded from [here](#).

```
1  /**
2   Outdoor Air Qual:
3   Name: outdoor_air
4   Purpose: This sk
5   (NO2, O3, AQI) fi
6   The data is sent
7   web interface or
8   color based on tl
9
10  @version 1.1 06/'
11  @modified by: Arc
12  */
13
14  // Include necessar
15  #include "Arduino_
16
17  // Automatically ge
18  #include "thingPro
19
20  // Set time interv
21  static const uint32
22  uint32_t lastReadT
23
24  // Sensor object fo
25  NiclaSenseEnv devic
26
27  /**
28   Initializes the s
```

The following sections will help you to understand the main parts of the example sketch shown before, which can be divided into the following:

Library imports

Data collection and Cloud
Updates
Cloud variables update

Library Imports

The necessary libraries for interacting with Nicla Sense Env board sensors and integrating with Arduino Cloud are included at the start.

```
1 // Include necessary I
2 #include "Arduino_Nicla
3
4 // Automatically gener
5 #include "thingProperties
```

In the code snippet shown before:

The `thingProperties.h` header file is included. Arduino Cloud generates this file to handle the synchronization of cloud variables. This allows sensor data (e.g., temperature, humidity, etc.) to be synchronized automatically with the Arduino Cloud dashboard for remote monitoring.

Cloud and Sensors Initialization

In the `setup()` function, we initialize the Nicla Sense Env onboard sensors and establish the connection to Arduino Cloud.

```
1 void setup() {
2   // Initialize serial
3   Serial.begin(9600);
4
5   // Short delay to wait
6   delay(1500);
7
8   // Initialize Cloud
9   initProperties();
10  ArduinoCloud.begin(AUTH_OTA);
11
12  // Debug information
13  setDebugMessageLevel(2);
14  ArduinoCloud.printDebug();
15
16  // Initialize Nicla Sense Env
17  if (device.begin()) {
18    Serial.println("Nicla Sense Env initialized");
19
20    // Enable the outdoor air quality sensor
21    device.outdoorAirQuality.enable();
22  } else {
23    Serial.println("Nicla Sense Env initialization failed");
24  }
25 }
```

In the code snippet shown
before:

The Arduino Cloud
connection is initialized
using `initProperties()`
and

`ArduinoCloud.begin()`.

These functions manage
the link between local
sensor variables and the
cloud.

Debugging is enabled with
`setDebugMessageLevel(2)`

, providing feedback on
the connection status.

The Nicla Sense Env
sensors are also initialized
and the outdoor air
quality sensor is enabled.

In the `loop()` function, sensor data is collected every 10 seconds and synchronized with the Arduino Cloud.

```
1 void loop() {  
2   // Update Arduino Cloud  
3   ArduinoCloud.update();  
4  
5   // Get the current time  
6   uint32_t currentTime = millis();  
7  
8   // Read sensors every 10 seconds  
9   if (currentTime - lastReadTime > 10000) {  
10    lastReadTime = currentTime;  
11    readSensors();  
12  }  
13 }
```

In the code snippet shown before:

The

`ArduinoCloud.update()`

function ensures that the Cloud connection is maintained and any updates to the sensor data are sent to the Arduino Cloud.

The `readSensors()`

function is called every 10 seconds to read sensor data and update the Cloud variables.

Cloud Variables Update

The `readSensors()` function reads data from the sensors and updates the Arduino Cloud variables. It also changes the RGB LED color of the Portenta C33 board based on AQI levels.

```

1 void readSensors()
2 // Get temperature
3 TemperatureHumidityPressure
4 OutdoorAirQualitySensor
5 RGBLED& rgbLED = (
6
7 // Check if sensor
8 if (tempHumSensor
9 // Read sensor c
10 float temperatu
11 float humidity :
12 float NO2 = airQ
13 float O3 = airQ
14 int airQualityIn
15
16 // Print sensor
17 Serial.print("-
18 Serial.print(tem
19 Serial.print("
20 Serial.print(hur
21 Serial.print(" S
22 Serial.print(NO2
23 Serial.print(" I
24 Serial.print(O3
25 Serial.print(" I
26 Serial.println(
27
28 // Update the oi

```

In the code snippet shown
before:

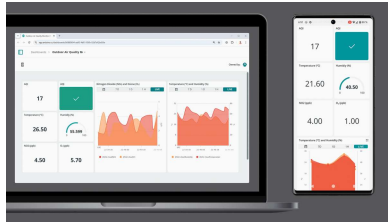
The RGB LED changes
color based on the AQI
value. Green ● for good,
yellow ● for moderate,
and red ● for unhealthy.

The cloud variables (`cloudTemperature` ,
`cloudHumidity` ,
`cloudNO2` , `cloudO3` ,
and `cloudAQI`) are
updated with the latest
sensor readings and sent
to Arduino Cloud for
remote monitoring. The
`cloudAQIAlert` variable
indicates whether the air
quality is good or
unhealthy.

The complete cloud example sketch can be downloaded [here](#).

Arduino Cloud Dashboard

The Arduino Cloud allows us to create a dashboard with professional real-time Human-Computer Interaction (HCI) elements, as seen in the following animation shows an active outdoor air quality monitor. **The animation has been sped up for illustrative purposes.**



Conclusions

In this application note, we explored how to integrate a Portenta C33 board with a Nicla Sense Env board to monitor environmental data such as temperature, humidity, NO₂, O₃, and AQI. We also demonstrated how Arduino Cloud allows for easy remote monitoring and real-time visualization of this data.

By using the Portenta C33, the Nicla Sense Env, and the Arduino Cloud, we can turn sensor readings into useful information. This setup makes it

feedback with the onboard RGB LED of the Portenta C33, and view the data remotely through a web dashboard or mobile app.

The main takeaway from this application is its potential for real-world use. By integrating IoT and Cloud technology, we can monitor air quality remotely, respond to environmental changes more quickly, and contribute to improving living conditions and sustainability efforts.

Next Steps

There are several opportunities to enhance the functionality of the outdoor air quality monitor:

- Integrate additional sensors to measure specific pollutants such as carbon monoxide (CO) or sulfur dioxide (SO₂).

- Optimize the monitor's energy efficiency to support long-term use in remote or outdoor locations.

- Create a custom dashboard on Arduino Cloud to track data trends over time and enable more in-depth air quality analysis.

- Develop an alert system that notifies users when pollutant levels exceed safe limits.

- Expand the system's use to different geographical areas to gather comparative air quality data

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

